

3. Automatikus kódoptimalizálás

Feladat azonosítója és címe: 3. Automatikus kódoptimalizálás

Beadó: Gyűrös Bence

Git repository: <https://github.com/BenceGyurus/code-optimizer>

Ennél a feladtnál azt próbáltam ki, hogy lehet-e LLM-eket nem csak sima kódgenerálásra használni, hanem egy mérhető kódoptimalizálási folyamat részeként. A lényeg az volt, hogy az AI ne csak „ránézésre” írjon át valamit, hanem először kapjon mérési adatokat, azok alapján válasszon célpontot, majd a módosítást a program tesztekkel és újraméréssel ellenőrizze. Így nem az számít, hogy a modell magabiztosan állítja-e, hogy gyorsított, hanem az, hogy a mérés ezt tényleg alátámasztja-e.

A programot a saját instrukcióim és döntéseim alapján készítettem, de a megvalósításban nagyrészt Codexet használtam. Ez sok kisebb körből állt: megfogalmaztam, milyen működést szeretnék, lefuttattam a programot, megnéztük az eredményeket, majd ezek alapján javítottunk rajta. A Codex főleg a kódolásban, refaktorálásban, promptok átírásában, shell scriptek elkészítésében és a dokumentáció összerakásában segített.

Közben két fő futtatási mód alakult ki. Az `optimizer run` egyetlen optimalizálási sessiont indít, vagyis egy modell egy prompt packkel megpróbál javítani egy projekten. Az `optimizer evaluate` ennél nagyobb: több modellt és több prompt pack-et futtat végig, majd ezek eredményeiből összehasonlítható mérési mátrixot készít. Nekem főleg ez volt érdekes, mert így nem csak egy-egy sikeres futást lehetett nézni, hanem azt is, hogy a különböző modellek és promptolási stratégiák mennyire stabilak.

A futások végén minden külön mappába kerül: az egyes sessionök, az összesített CSV és YAML eredmények, a Markdown riport és a diagramok. A runtime és a hardveres mérések nem egyetlen futásból jönnek, hanem több ismétlés átlagából. A végső méréseknél például 15 futásból számoltam átlagot, mert egyetlen benchmark futás könnyen félrevezető lehet.

A program működése

Az egész program arra épül, hogy az AI ne közvetlenül, kontroll nélkül írja át a kódot. Inkább döntéstámogató szerepet kap egy olyan folyamatban, ahol előbb mérés van, és csak utána módosítás. A modell baseline futási időt, teszteredményt, profilozási adatokat, hardveres metrikákat és kódrészleteket kap. Ezek alapján kell kiválasztania, hol érdemes hozzányúlni. Ehhez a tool-okat használ a program, ezek közül ő választja ki, hogy melyeket szeretné használni bizonyos kereteken belül.

Ez azért fontos, mert optimalizálásnál könnyű rossz irányba indulni. Egy modell tud nagyon meggyőzően javasolni egy átírást, de attól még lehet, hogy a kód lassabb lesz, vagy megváltozik a viselkedése. Ezért a program nem azt várja, hogy a modell minél több fájlt írjon át, hanem azt, hogy találjon egy konkrét szűk keresztmetszetet, és arra adjon egy lehetőleg kicsi, ellenőrizhető javítást.

Az optimalizálásnál pár egyszerű szabályt próbáltam követni. Először algoritmikus javítást érdemes keresni, mert egy $O(n^2)$ megoldás $O(n)$ -re cserélése általában többet ér, mint pár apró lokális gyorsítás. A változtatás legyen kicsi és célzott, ne írja át feleslegesen az egész programot. A viselkedés nem változhat meg, ezért minden patch után teszt fut. A legfontosabb mérőszám a runtime, a cache, L1, branch és LLC adatok pedig inkább magyarázó szerepet kapnak. Ez azért fontos, mert lehet, hogy a cache hit-rate kicsit rosszabb lesz, de a program mégis sokkal gyorsabb, ha közben jóval kevesebb munkát végez.

A modell nem kapja meg a teljes nyers logot, mert az gyorsan túl sok token lenne, főleg kisebb modelleknél. Ehelyett rövidített, strukturált adatokat kap: tesztösszefoglalót, benchmark átlagot/minimumot/maximumot, profilozási hotspotokat, hardveres metrikákat, aktuális állapotot, engedélyezett toolokat és célzott kódrészleteket. Többfájlos projektnél ezt kiegészíti a fájllista, a függvényvázlat és a szűkebb kódkontextus.

Ezzel az volt a cél, hogy a modell ne egy nagy, rendezetlen szöveget kapjon, hanem egy mérési csomagot. Így könnyebb eldöntenie, hogy mi történt eddig, milyen lépés következhet, és melyik kódrészlethez érdemes nyúlni. Ez a kisebb modelleknél különösen fontos volt, mert ők hamarabb elvesznek a túl sok kontextusban.

A kimenetből nem csak az derül ki, hogy sikerült-e egy futás. Látszik a patch, a tesztek eredménye, a baseline és az optimalizált futási idő, illetve a speedup is. Az `evaluate` futás ezekből készít összesített CSV és YAML fájlokat, `report.md`-t és diagramokat. Ezekből már össze lehet hasonlítani a modelleket, prompt packeket, toolhasználatot, LLM hívásszámot és a hardveres metrikák változását is.

Nálam ez volt a projekt egyik fontos része: nem csak az érdekelt, hogy egy modell egyszer tud-e gyorsítani, hanem az is, hogy ezt mennyi próbálkozással, mennyi toolhívással, mennyire stabilan és milyen promptolási stratégiával teszi meg.

A CLI megkapja a projektet, a modellt vagy modelllistát, a prompt packeket, illetve a teszteléshez és méréshez használt parancsokat. Ezután minden optimalizálási próbához külön workspace készül a `results/` mappában. Ez egy biztonsági réteg: az AI nem az eredeti fájlokon dolgozik, hanem egy másolaton. Ha a patch rossz, lassabb, vagy elrontja a teszteseteket, az eredeti projekt nem sérül.

A session elején a program betölti az adott prompt pack-et. Ebben külön fájl van az általános szerepre, a döntésre, az elemzésre, a patch írására és az eredmény értékelésére. A modell minden lépésnél megkapja az aktuális állapotot, az engedélyezett műveleteket, a korábbi toolok rövidített eredményét és a mérési adatokat. Válaszként strukturált JSON-t kell adnia, például azt, hogy `run_baseline`, `profile_execution`, `propose_change` vagy `apply_and_verify` legyen a következő lépés. Így nem szabadon beszélget, hanem a program keretein belül dönt.

Az első tényleges lépés a baseline mérés. Ez adja meg, hogy a kód milyen gyors és milyen hardveres mutatókat produkál az optimalizálás előtt. Ilyenkor lefutnak a tesztek, a benchmark, és ha be van állítva, akkor a hardveres profilozás is. A tesztek több ismétléssel futnak, de a modell ebből csak összefoglalót kap: hány futás ment át, mennyi volt az átlagos futásidő, volt-e hiba, illetve mennyi lett a benchmark átlaga. A `perf` kimenetnél ugyanez történik, csak ott a cache, branch és L1 jellegű adatokból számol átlagot a program.

Ezután jön a profilozás és a célpont kiválasztása. Egyfájlos projektnél ez elég egyszerű, többfájlosnál viszont nem akartam az egész projektet egyben belerakni a promptba. Ilyenkor a program inkább fájllistát, függvényvázlatot, profilozási részleteket és célzott kódrészleteket ad. Ettől a kisebb modellek kevésbé vesznek el a nagy kontextusban, a nagyobbak pedig jobban tudnak a tényleges hotspotra figyelni.

Ha megvan a célpont, a modell a `propose_change` lépésben patch-et javasol. Ezt a program nem fogadja el azonnal. Először eltárolja, majd az `apply_and_verify` lépésben alkalmazza a workspace-ben, lefuttatja a teszteseteket, és csak akkor megy tovább, ha a kód helyes maradt. Ha a tesztesetek elbuknak, rollback történik. Ez több futásnál is hasznos volt, mert volt olyan modell, amelyik jó irányba indult, de egy apró szemantikai hibával már elrontotta volna a programot.

Ha a patch átment a teszteseteken, a program újraméri az optimalizált kódot. Ugyanazok a benchmark és hardveres mérési parancsok futnak, ugyanannyi ismétléssel, mint baseline-nál. Ez azért fontos, mert így az előtte-utána összehasonlítás nem két külön módszerből jön, hanem ugyanabból a mérési folyamatból. Az `evaluate_result` ezután összeveti a két állapotot. Ha a gyorsulás elég jó, a session `DONE` állapotban zárul. Ha nincs javulás, vagy elfogyott a budget, akkor a futás megáll, és sikertelen vagy nem elég jó optimalizációként kerül a riportba.

Az `evaluate` mód ugyanezt ismétli végig több modellre és prompt packre. Minden kombináció külön session, a végén pedig készül egy összesítés. Innen jön az `aggregated_results.csv`, az `aggregated_results.yaml`, a `report.md` és a `charts/` mappa. Ezekből lehet később megnézni, melyik modell mennyi toolt használt, hány LLM hívás kellett neki, sikeres volt-e a patch, mekkora gyorsulást ért el, és hogyan változtak a hardveres mutatók.

Külön figyelni kellett arra is, hogy a modell ne tudjon végtelen ciklusba kerülni. Ehhez egy állapotgépet használtam: minden állapotban csak bizonyos toolok hívhatók meg. Baseline mérés előtt például nem lehet patch-et alkalmazni, patch után pedig kötelező a tesztelés. Emellett minden session kap limitet az LLM hívásokra, tool hívásokra és iterációkra (`--max-llm-calls`, `--max-tool-calls`, `--max-iterations`). Erre azért volt szükség, mert a korai próbáknál volt olyan modell, ami ugyanazt a lépést kérte újra és újra, vagy hibás JSON-t adott vissza. Ilyenkor a rendszer nem vár örökké, hanem megállítja a futást, rollbackel, vagy `DONE / FAILED` állapotba teszi a sessiont.

A rendszer fő eszközei

Tool	Mire való
<code>inspect_codebase</code>	A modell kontextust kap a projektről, a fájlokról és a releváns kódrészletekről.
<code>run_baseline</code>	Lefuttatja az alap teszteseteket, a benchmarkot és a hardveres méréseket.
<code>profile_execution</code>	cProfile és Linuxon <code>perf</code> alapján megmutatja, hogy hol megy el az idő.
<code>analyze_candidate</code>	A modell kiválasztja, melyik függvényt vagy fájlt érdemes optimalizálni.
<code>propose_change</code>	A modell patch-et javasol a kiválasztott célpontra.

Tool	Mire való
apply_and_verify	A program alkalmazza a patch-et, majd lefuttatja a teszteket.
remeasure	Újraméri az optimalizált kódot, hogy össze lehessen hasonlítani a baseline-nal.
evaluate_result	Eldönti, hogy az eredmény elfogadható-e, vagy nem érdemes tovább próbálkozni.
rollback_to_checkpoint	Hibás vagy lassabb módosításnál visszaállítja a biztonságos állapotot.

Több promptolási stratégia is van a `prompts/` mappában. Ezek ugyanazt a programot használják, csak más irányból próbálják rávezetni a modellt a jó döntésre. Ez azért volt hasznos, mert így nem csak modelleket lehetett összehasonlítani, hanem azt is, hogy ugyanaz a modell más promptolási technikával máshogy viselkedik-e.

A `zero_shot` minimális instrukcióval indul, a `few_shot` példákkal mutatja meg a várt működést, a `knowledge_gen` előbb háttértudást épít, a `hardware_focus` jobban figyel a cache és branch jellegű mérésekre. A `self_refine` saját ellenőrzést kér a modelltől, a `cot` és a `least_to_most` lépésenkénti gondolkodást használ, a `prompt_chaining` pedig jobban szétszedi a döntési lépéseket. Az `agentic` és a `hypothesis_driven` már kifejezetten azt hangsúlyozza, hogy a modellnek a program állapotához és a mért adatokhoz kell igazodnia.

Kiemelt prompt pack: `hypothesis_driven`

A mérések alapján a `hypothesis_driven` volt az egyik legjobban teljesítő prompt pack, ezért ezt emeltem ki. Itt a modellnek nem elég annyit mondania, hogy „ezt optimalizálnám”. Minden lépésnél kell egy mérhető hipotézis: melyik metrika fog javulni, miért pont azt a kódrészletet választja, és a patch miért segít ezen.

Ez jól passzolt a programhoz, mert itt amúgy is minden döntés méréshez van kötve. Ha a modell például azt állítja, hogy egy ciklust érdemes átírni, akkor meg kell neveznie, milyen mérhető hatást vár ettől. Utána a program tényleg újraméri a kódot, tehát kiderül, hogy a hipotézis működött-e vagy sem.

Prompt fájl	Mire való
master.md	Beállítja a modell szerepét, az állapotgépet, az engedélyezett műveleteket és azt, hogy mérés alapján kell dolgoznia.
decision.md	Minden lépésnél kiválasztja a következő toolt, de csak érvényes JSON választ adhat.
analyze_candidate.md	Kiválaszt egy konkrét hotspotot, és ehhez mérhető hipotézist kell adnia.
propose_change.md	Patch-et kér a célpontra, rövid indoklással és várható hatással.
evaluate_result.md	Összeveti az elvárt és a mért eredményt, majd eldönti, hogy meg kell-e állni.
config.yaml	A prompt pack neve, verziója és rövid leírása.

Nem egyetlen nagy promptot használtam, hanem prompt packeket. Ennek az volt az oka, hogy a program nem egy darab választ vár a modelltől, hanem egy folyamatot vezérel. Más szöveg kell akkor, amikor csak a következő toolt kell kiválasztani, más akkor, amikor célpontot elemez, más akkor, amikor patch-et ír, és más akkor is, amikor az eredményt értékeli.

Ha ez egyetlen nagy promptban lenne, sokkal könnyebben összemosódnának a szerepek. A modell egyszerre próbálna stratégiai döntést hozni, kódot írni és eredményt értékelni, ami a korai futásoknál sok hibát okozott. A prompt packes felépítésnél viszont ugyanaz a rendszerlogika marad, de külön lehet finomítani a döntési, elemzési, patch-generálási és értékelési promptot.

A `hypothesis_driven` prompt pack konkrét promptjait raktam a dokumentációban is. A `{{...}}` részek sablonváltozók, ezeket futás közben tölti ki a program az aktuális állapottal, mérésekkel, tool listával és kódkontextussal.

- master.md
- decision.md
- analyze_candidate.md
- propose_change.md
- evaluate_result.md

Teszt kódok, futtatók és promptok

Két fő tesztprogrammal dolgoztam. Az első a [heavy_compute.py](#), ami egy egyfájlos, direkt számításigényes Python program saját unittest tesztekkel. Ezen jól látszott, hogy a modellek megtalálják-e az egyértelmű algoritmikus hibát, és tudnak-e úgy gyorsítani rajta, hogy a tesztek továbbra is átmenjenek.

A második a [complex_pipeline](#), ami már több fájlból áll. Ehhez külön tesztfájl is tartozik: [test_market_sim.py](#). Ezt azért tettem mellé, mert a valódi projektekben általában nem egyetlen fájlban kell gondolkodni. Itt már fontosabb volt, hogy a program hogyan ad kontextust a modellnek, és hogy a modell meg tudja-e különböztetni a valódi lassú részt attól a függvénytől, ami csak meghívja azt.

A Debianos futtatásokhoz külön shell scriptek készültek. Ezek főleg arra jók, hogy ne kézzel kelljen mindig újra megadni a modelleket, prompt packeket, `perf` countereket, ismétlésszámokat és benchmark parancsokat. Így kisebb az esélye annak, hogy két futás azért lesz eltérő, mert közben rosszul lett bemásolva egy parancs.

Script	Mire használtam
evaluate_debian_smoke.sh	Gyorsabb próbamérésre, hogy egy olcsóbb vagy kisebb futással kiderüljön, működik-e a teljes pipeline.
evaluate_debian_full.sh	Az egyfájlos heavy_compute.py teljesebb mérésére, több modellel és több prompt pack-kel.
evaluate_debian_complex.sh	A többfájlos complex_pipeline mérésére, a legjobban teljesítő modellekkel és prompt stratégiával.
repeat_unittest_summary.py	Nem shell script, de a futatók ezt használják arra, hogy a tesztek több ismétlésének eredményéből rövid összefoglaló készüljön.

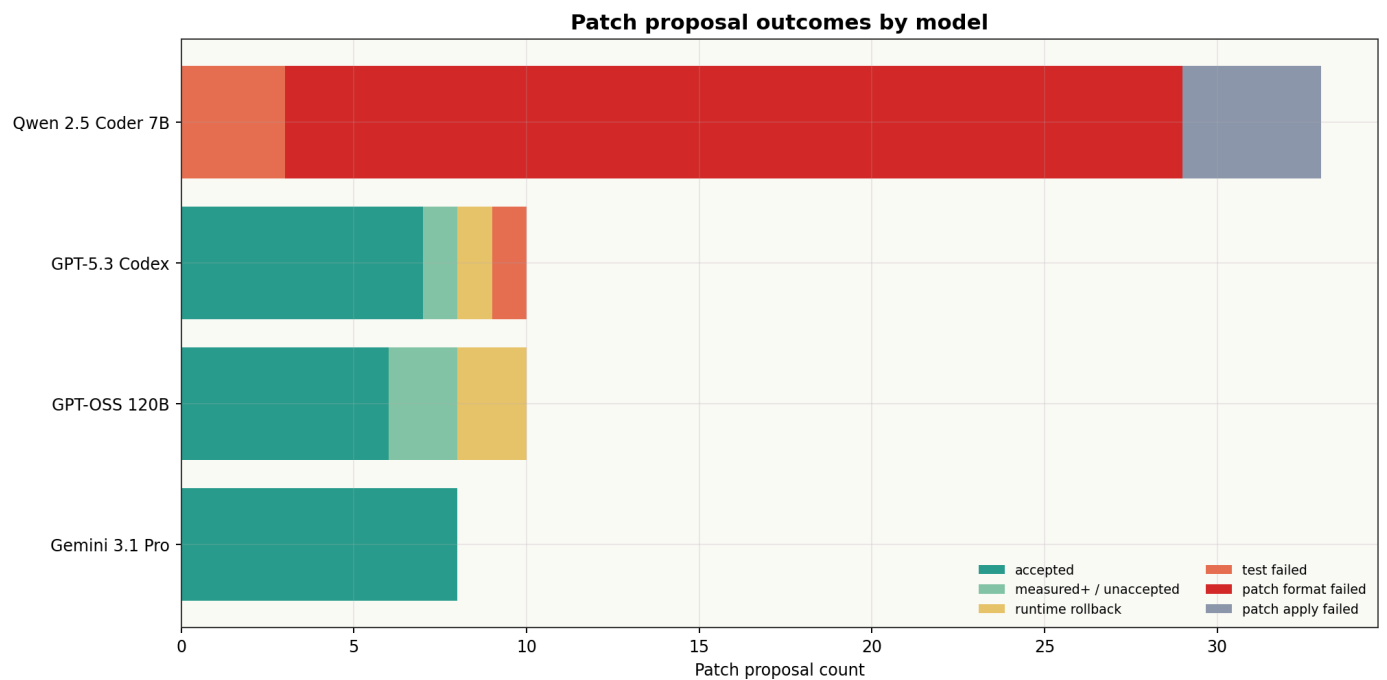
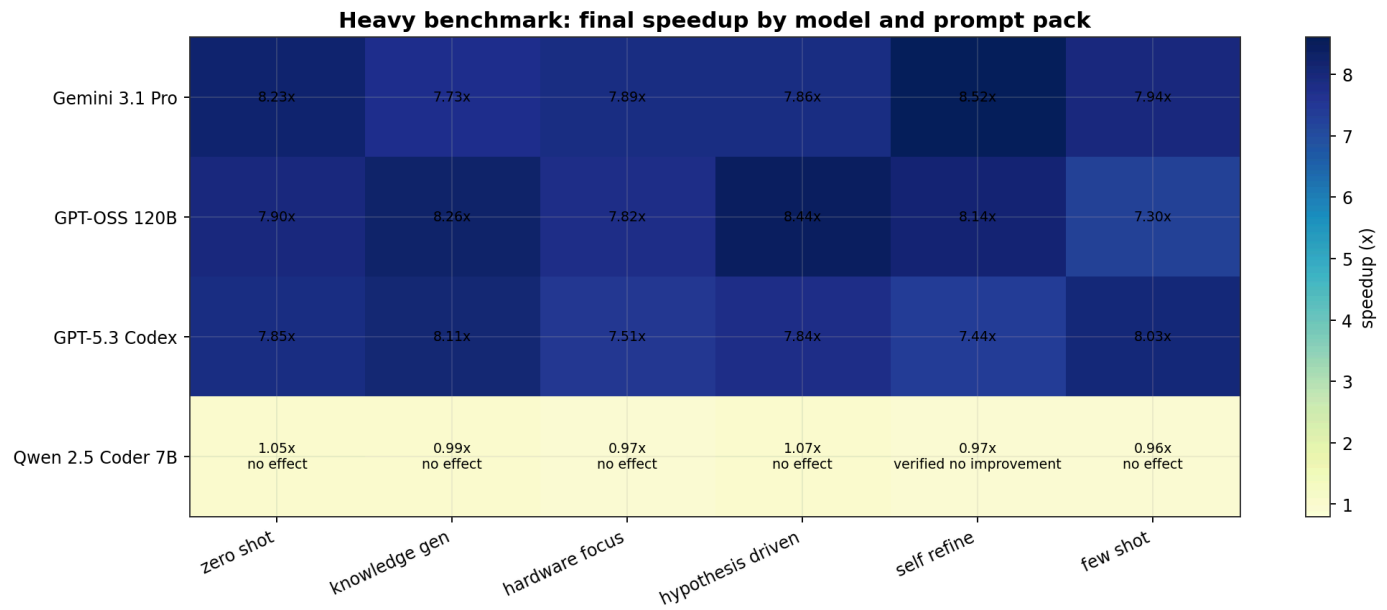
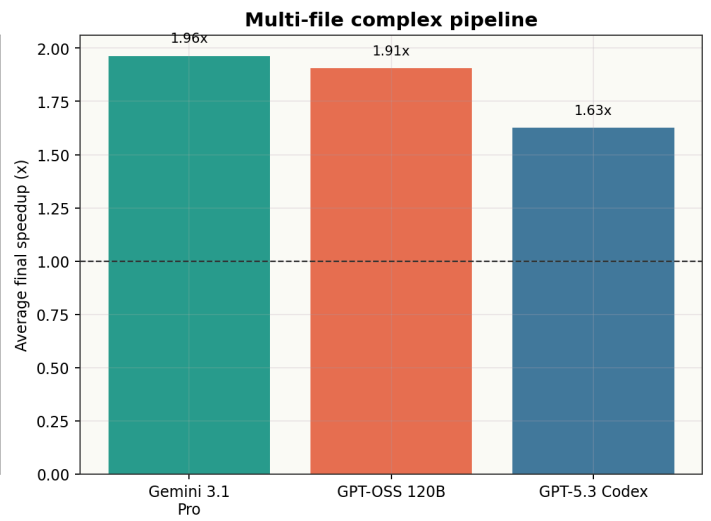
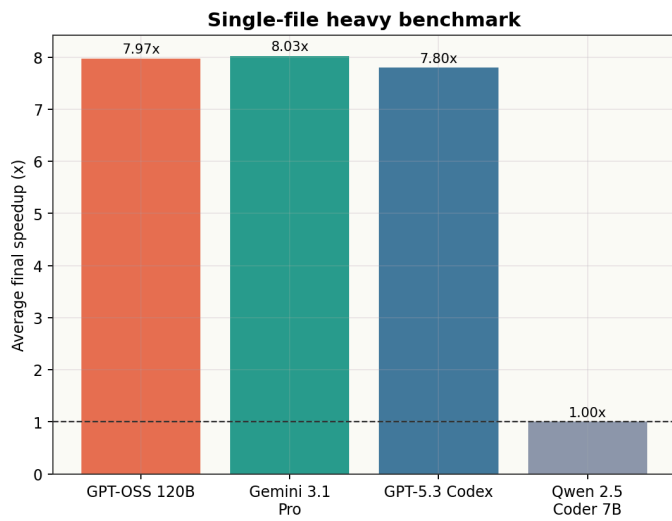
A promptok a [prompts](#) mappában vannak. A heavy mérésben főleg ezek szerepeltek: [zero_shot](#), [knowledge_gen](#), [hardware_focus](#), [hypothesis_driven](#), [self_refine](#) és [few_shot](#). A komplex többfájlos mérésnél már a legerősebb modelleket futtattam a [hypothesis_driven](#) prompt packkel, mert az előző eredmények alapján ez passzolt legjobban ehhez a mérésvezérelt működéshez. A részletesebb, diagramokkal alátámasztott feldolgozás a [debian-combined-llm-prompt-analysis](#) mappában van, a heavy futás külön elemzése pedig itt: [debian-full-heavy-1778099536167691431](#).

Eredmények

A teszteket Debian 12 alatt futtattam, mert a hardveres mérésekhez Linuxos `perf` countereket használtam. Az OpenRouteres modellek külső szolgáltatáson keresztül mentek, a self-hostolt Qwen 2.5 Coder 7B viszont helyben futott, egy NVIDIA P104-100 videokártyán, 8 GB VRAM-mal.

A végső teszteknel kétféle kód szerepelt. A `heavy_compute.py` egyfájlos, erősen számításigényes példa volt. Itt a legerősebb modellek szinte mindig a `segmented_prefix_sums_slow` részt találták meg. Ebben a függvényben az volt a lényeg, hogy a program újra és újra kiszámolt olyan részösszegeket, amelyeket futó összeggel sokkal olcsóbban is lehetett volna kezelni.

Ezt a modellek általában $O(n^2)$ jellegű megoldásból $O(n)$ megoldássá alakították. Emiatt több modellnél is 7-8x körüli gyorsulás jött ki. Ez elsőre gyanúsán egységesnek tűnhet, de ebben az esetben inkább az látszik belőle, hogy volt a kódban egy nagyon domináns, könnyen mérhető hotspot. Ha minden jó modell ugyanazt a nagy hibát találja meg, akkor a gyorsulásuk is hasonló lesz.



A második méréshez Codexszel generáltattam egy többfájlos `examples/complex_pipeline` példát is. Ezt azért raktam bele, mert egy többfájlos programnál már nem olyan egyértelmű, hogy hol kell optimalizálni. Sokszor van egy magasabb szintű függvény, ami csak összefogja a folyamatot, és meghív több kisebb függvényt.

A profilozó ilyenkor nagy kumulatív időt mutathat erre a felső függvényre. A kumulatív idő azt jelenti, hogy nem csak az adott függvény saját futásideje számít bele, hanem azoké a függvényeké is, amelyeket közben meghív. Ezért nem biztos, hogy maga a felső függvény a lassú. Lehet, hogy ő csak továbbhív egy ténylegesen drága kisebb függvényt.

Emiatt a jobb optimalizációhoz nem elég azt nézni, hol gyűlik össze a legtöbb idő. Azt is meg kell keresni, melyik konkrét függvény végzi a sok munkát. Erre példa volt a `customer_recent_totals`, ahol már nem a teljes pipeline-t kellett piszkálni, hanem azt a kisebb részt, ami valóban sok számítást végzett. Így az LLM is célzottabb kontextust kapott, és nem az egész többfájlos projektből kellett kitalálnia, hol van a valódi probléma.

A modelleknel nem az volt a legnagyobb különbség, hogy ki talált teljesen más ötletet. Inkább az számított, ki tudta stabilan végigvinni ugyanazt a jó mérnöki döntést. A Gemini 3.1 Pro, a GPT-OSS 120B és a GPT-5.3 Codex jól működtek a rendszerrel. A self-hostolt Qwen 2.5 Coder 7B sokszor közel volt a jó irányhoz, de a patch formátuma vagy a kód szemantikája nem volt elég stabil. Ez nekem azért volt érdekes, mert egy lokális vagy olcsóbb modell nem biztos, hogy a teljes folyamatban is olcsóbb, ha közben sok sikertelen próbálkozást eredményez.

Futtatás

Fejlesztés közben a CLI telepítés nélkül is futtatható. Ez főleg gyors ellenőrzésre jó, például arra, hogy a függőségek és a konfiguráció rendben vannak-e:

```
PYTHONPATH=src .venv/bin/python -m optimizer.cli doctor
```

Egy olcsó smoke teszt Debianon. Ezzel érdemes kezdeni, mert csak azt ellenőrzi, hogy a teljes pipeline végig tud-e menni, mielőtt drágább modellmátrixot futtatnék:

```
OPENROUTER_API_KEY=sk-or-... ./scripts/evaluate_debian_smoke.sh examples/complex_pipeline results/debian-smoke-complex
```

A komplex, többfájlos végső futás már a komolyabb mérés. Itt több ismétlés, több mérés és a kiválasztott modellek/promptok futnak:

```
OPENROUTER_API_KEY=sk-or-... RUN_REPETITIONS=15 ./scripts/evaluate_debian_complex.sh
```

Tanulságok

A legfontosabb technikai tanulság az volt, hogy az AI-alapú kódoptimalizálás akkor kezd használható lenni, amikor már nem csak promptolásból áll. Egy modell tud jó ötletet mondani, de ettől még nem biztos, hogy a kód helyes marad, vagy tényleg gyorsabb lesz. Ezért kellett köré mérés, tesztelés, rollback és újramérés. A modell javasol, de az elfogadásról már a program dönt.

Ezt főleg a `gpt-oss-120b` próbálgatásánál láttam. Többször előfordult, hogy a modell sok LLM hívást és toolhívást elhasznált, de közben nem jutott el érdemi optimalizáláshoz. Ilyenkor nagyon fontos volt, hogy legyen felső limit, mert különben egy ilyen futás könnyen végtelennek tűnő próbálkozássá válhatna.

A modellek összehasonlításánál ezért nem csak a végső speedup számít. Egy egyszerűbb benchmarkon több modell is ugyanarra a 7-8x gyorsításra jutott, mert a legnagyobb hiba nagyon egyértelmű volt. Ilyenkor inkább az dönt, mennyi hibás JSON-t adnak, mennyire pontos a patch, és mennyire tartják be az állapotgép szabályait. Nem csak az a kérdés, hogy megtalálta-e az ötletet, hanem az is, hogy végig tudta-e vinni elfogadható formában.

A promptstratégiáknál azok működtek jobban, amelyek tényleg illeszkedtek a program logikájához. A `hypothesis_driven` azért volt jó, mert ugyanarra kényszerítette a modellt, amit a rendszer is elvárt: mérhető állítás, konkrét célpont, ellenőrzött patch, majd döntés az eredmény alapján. Egyszerű esetben a `zero_shot` is elég lehet, de többfájlos vagy bizonytalanabb projektnél sokkal fontosabb, hogy a prompt ne csak általános tanács legyen, hanem értse a toolokat és az állapotokat.

A hardveres méréseknél érdekes volt, hogy a cache hit-rate önmagában nem elég jó sikerességi mutató. Volt olyan futás, ahol a cache arány kicsit romlott, de a program sokkal gyorsabb lett, mert egyszerűen kevesebb munkát végzett. Emiatt nálam a runtime maradt az elsődleges metrika, a cache, L1, branch és LLC adatok pedig inkább magyarázó adatok lettek. Ezek segítenek megérteni, mi változott a hardver szintjén, de nem írják felül azt, hogy a program ténylegesen gyorsabb lett-e.

A többfájlos példánál az jött ki, hogy a profilozási eredményt nem szabad vakon olvasni. A cProfile kumulatív ideje néha olyan függvényre mutat, ami csak meghív sok másik drága részt, de nem ott kell optimalizálni. Ilyenkor több kontextus kell: látni kell a fájlok közötti kapcsolatot, a hívási láncot, és azt is, melyik kisebb függvény tölti ténylegesen számolással az időt. Emiatt került bele később az is, hogy nagyobb projektnél ne egyben kapja meg a modellt az egész kódot, hanem a program logika alapján tördelje és célzottabban adja oda.

Az egyik legnagyobb fordulat akkor jött, amikor a rendszer már nem csak a kód szerkezete alapján próbálta eldönteni, hol érdemes optimalizálni, hanem külön függvény szintű profilozást is kapott. A korábbi verzióban a modell sokszor azt a részt választotta, ami ránézésre bonyolultnak tűnt: sok ciklus volt benne, hosszabb volt, vagy hardveresen érdekesebbnek látszott. Ez viszont nem mindig jelentette azt, hogy a program futásidejének nagy része tényleg ott ment el. Emiatt több futás csak nagyon kis javulást hozott, például 1.02x körülit, mert a modell nem a valódi szűk keresztmetszetet javította.

Ezután bekerült egy cProfile alapú mérés, ami már függvényenként megmutatta, hogy futás közben hol gyűlik össze a legtöbb idő. Fontos volt, hogy ebből sem a teljes nyers profiler kimenetet kapta meg az LLM, hanem csak egy rövidített hotspot listát. Így a kontextus nem lett feleslegesen nagy, de a modell mégis sokkal jobb információból tudott dönteni. A `segmented_prefix_sums_slow` például így került előtérbe, mert a mérés alapján ez vitte el a futásidő nagy részét, nem azok a függvények, amelyek csak statikusan néztek ki bonyolultnak.

Ennek nagyon látványos hatása volt. Ugyanazzal a modellel és ugyanazzal a promptolási stratégiával a korábbi minimális javulás helyett már 7-8x körüli gyorsulás jött ki. Itt nem arról volt szó, hogy a prompt hirtelen sokkal jobb lett, hanem arról, hogy a modell végre jó adatból dolgozott. Ebből tanulság volt, hogy az LLM teljesítménye nem csak a modellen és a prompton múlik, hanem azon is, hogy milyen mérési eredményeket és milyen formában adunk be neki.

Összességében nekem az lett a feladat fő tanulsága, hogy az AI önmagában még nem elég egy jó optimalizáló rendszerhez, főleg kisebb vagy kevésbé stabil modelleknél. A jó eredményt nem csak a modell adja, hanem az, hogy van körülötte mérés, guardrail, teszt, rollback, budget és riportolás. Így már nem csak az látszik, hogy az AI átírt valamit, hanem az is, hogy mit változtatott, miért, hányszor próbálkozott, sikeres volt-e, és mennyit gyorsult tőle a program.

Részletesebb elemzés és további diagramok: [docs/experiments/debian-combined-llm-prompt-analysis/README.md](https://github.com/GoogleCloudPlatform/llm-prompt-analysis/blob/main/docs/experiments/debian-combined-llm-prompt-analysis/README.md)

Technikai használati útmutató a CLI-hez és a régi részletesebb architektúrális leíráshoz: [USERMANUAL.md](https://github.com/GoogleCloudPlatform/llm-prompt-analysis/blob/main/USERMANUAL.md)

A program elkészítéséből származó tapasztalataim

A program elkészítésénél több modellt és több felületet is kipróbáltam: webes ChatGPT-t, CLI-s eszközöket és desktopos Codexet is. A tervezéshez főleg a webes ChatGPT-t használtam GPT-5.4 modellel, mert nagyobb kontextusban kényelmesebb volt végiggondolni a teljes követelményrendszert. Először csak felvázoltam, milyen programot szeretnék, hogyan működjön, milyen mérések legyenek benne, és hogyan akarom összehasonlítani a modelleket. A végén ebből írtam vele egy összefüggőbb követelményrendszert.

Ezt először GeminiCLI-nak adtam oda, automatikus modellválasztással, ahol Gemini 3.1 Pro-t és Gemini 3 Flash-t használt. Itt az volt az érzésem, hogy a feladat egyben túl nagy volt. A rendszernek csak egy részét valósította meg, és azok sem mindig úgy működtek, ahogy szerettem volna. Később ugyanazt a követelményrendszert Codexnek adtam be, GPT-5.4 és GPT-5.5 modellekkel. A Codex először checklistet készített, majd csinált egy demó programot. Ez az első verzió még messze nem volt kész, de legalább futott, és innentől volt mit mérni, javítani és továbbépíteni.

Ebből nekem az jött le, hogy nagyobb fejlesztési feladatnál sokkal jobb először tervet kérni, és csak utána implementációt. Ha a modell előbb megtervezi a megoldást, én azt átnézem és pontosítom, akkor kevesebb félreértés lesz. Amikor rögtön nagy, több részből álló feladatot adtam be, gyakrabban kihagyott részeket, rossz helyen javított, vagy nem azt tekintette fő feladatnak, amit én. Amikor külön választottam a tervezést, megvalósítást és ellenőrzést, sokkal kevesebbet kellett utólag javítani.

A korai, még nem jól működő iránynál az is kiderült, hogy az Opencode-ban használt ChatGPT-5.3 Codex (Github Copilot pro) könnyen elveszti a fonalat, ha egyszerre túl sok szerepet kap. Próbáltam úgy dolgozni vele, hogy közben vezessen jegyzőkönyvet

a tapasztalatokról, készítsen tesztprogramokat, és magát az optimalizáló keretrendszert is fejlessze. Ilyenkor sokszor összekeverte, mi az aktuális fő feladat. Ha egy hibát a programban akartam javítani, néha úgy vette, hogy a tesztet kell átírni, vagy csak a jegyzőkönyvbe kell beírni a problémát. Ebből az jött le, hogy ilyen projektnél jobb tisztábban szétválasztani a feladatokat, vagy külön agenteket használni külön szerepekre.

Ugyanebben a korai szakaszban az egységtesztek hibáinak elemzése sem ment mindig jól. Amikor optimalizálás után nem futottak le a tesztek, gyakran először a tesztek átírását javasolta. Nem azt nézte, hogy a program módosítása rontotta-e el a viselkedést. Csak akkor ment jó irányba, amikor nagyon pontosan leírtam, hogy nem a tesztet kell hozzáigazítani a hibás kódhoz, hanem a kódot kell úgy javítani, hogy a meglévő tesztek maradjanak érvényesek. Volt olyan is, hogy olyan módszert talált ki, amitől a program lefutott, de valójában nem végzett érdemi LLM-alapú vagy heurisztikus optimalizálást. Itt még más volt az eredeti terv, mert akkor a heurisztikus optimalizálás is erősebben szerepelt volna.

Az is előfordult, hogy a modell kerülő megoldást választott, hiába kértem, hogy ne tegye. Például kértem, hogy ne új flag bevezetésével kerülje meg a hibát, hanem magát a hibás működést javítsa ki, mégis új flaget hozott létre. Ezért lett később fontosabb a guardrail, a pontos tool szerződés és a verifikáció. A végső programban már nem elég az, hogy a modell mond valamit: a patch csak akkor számít sikeresnek, ha a tesztek és az újramérés alapján is elfogadható.

Ellenőrzés

```
PYTHONPATH=src .venv/bin/python -m compileall -q src/optimizer
PYTHONPATH=src .venv/bin/python -m pytest -q
PYTHONPATH=src .venv/bin/python -m optimizer.cli doctor
```